

# Modern Row Limiting

Evolving Oracle Top-N Queries from Legacy Patterns to Modern SQL  
Standards



---

# The Hook

Why readability is the ultimate performance metric in modern database engineering.



# Problem Scenario: The "Russian Doll" SQL

---

## Readability Debt

Nested subqueries obscure business logic, making SQL maintenance a high-risk activity for complex reporting systems.

## Pagination Pain

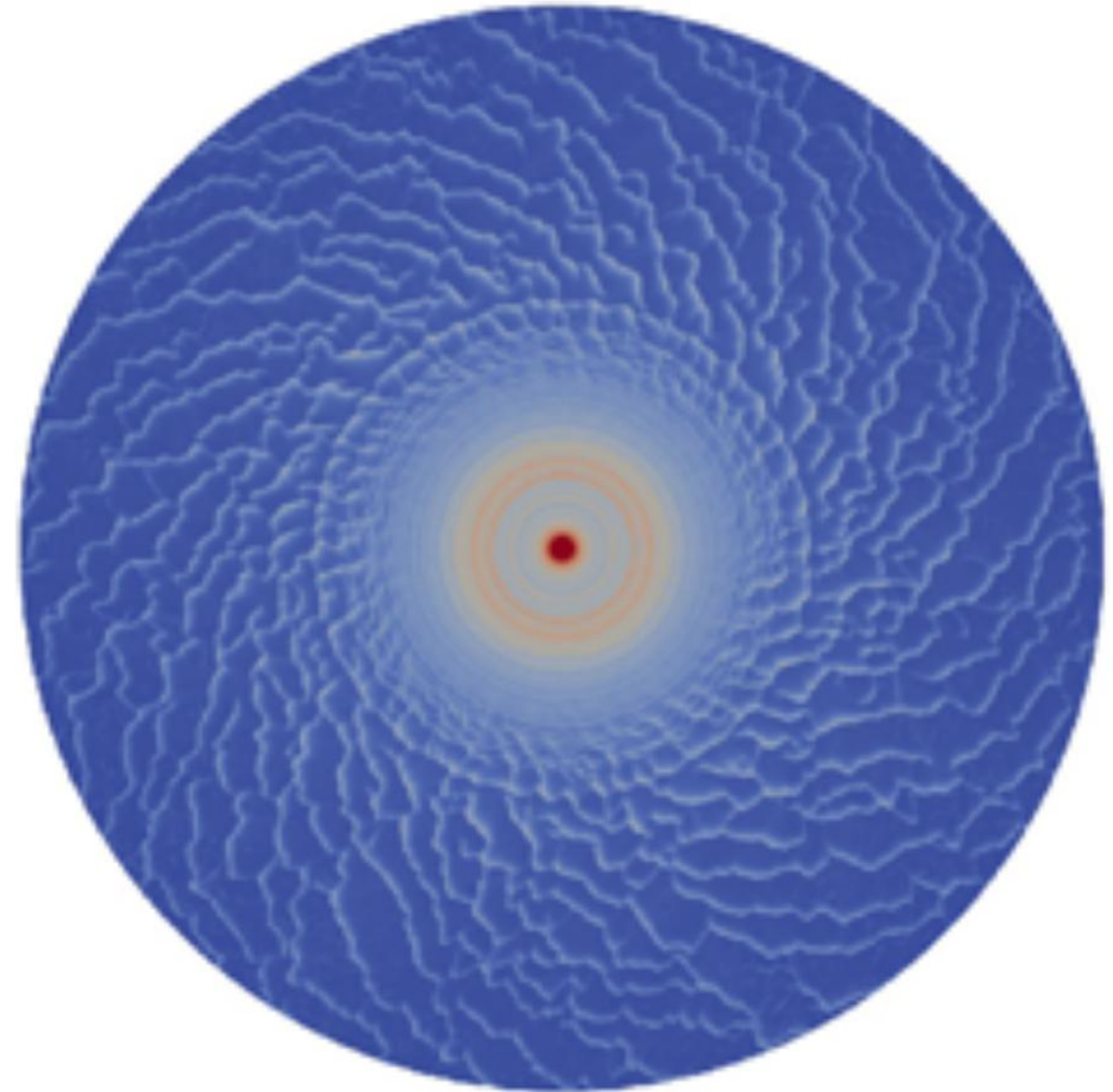
Traditional ROWNUM pagination requires triple-nesting and fragile alias management to prevent logic collisions.



# Visualizing Technical Complexity

Legacy patterns force developers to focus on **filtering mechanics** rather than **business intent**.

This adds significant cognitive load, slows down code reviews, and increases the likelihood of "Off-by-One" errors in data retrieval.





---

# The Solution

The ANSI-Standard `row_limiting_clause`: Cleaner, Faster,  
Standardized.



# Modern Syntax Components

---



## OFFSET

Skip N rows effortlessly. Perfect for clean, stateless pagination across web applications.



## FETCH FIRST

Clearly declare counts or percentages. The Optimizer treats this as a first-class intent.



## WITH TIES

Natively handle rows with identical sort values without extra analytical functions.



# Performance Consistency

---

ROWNUM Legacy

12ms

FETCH FIRST

12ms

Execution plans show identical **STOPKEY** optimizations. The win is in **code simplicity**.



# Top-N Feature Comparison

| Feature             | Legacy (ROWNUM)          | Modern (FETCH FIRST) |
|---------------------|--------------------------|----------------------|
| Standard Compliance | No (Oracle Proprietary)  | Yes (ANSI SQL:2008)  |
| Code Complexity     | High (Nested Subqueries) | Low (Single Clause)  |
| Percentage-Based    | Manual Calculation       | Native Support       |
| Tie Management      | DENSE_RANK required      | Native (WITH TIES)   |



---

# Demo Walkthrough

Benchmarking declarative syntax on a 1,000,000 record dataset.



# Key Insights & Pitfalls

---

- ✔ **Order By is Mandatory:** Top-N is non-deterministic without a valid sort key.
- ⚠ **Offset Cost:** High offsets still scan preceding rows ( $O(N)$  complexity).
- 📈 **Dashboard Utility:** Use PERCENT for dynamic analytical visualizations.
- ⌘ **Internal Transformation:** FETCH FIRST is rewritten to STOPKEY operations automatically.



# The Benefit: ROI

---

50%

Less Code Complexity

## Streamlined Development

Standardizing on FETCH FIRST reduces the surface area for bugs and accelerates the onboarding of new developers to the database schema.



# Questions?

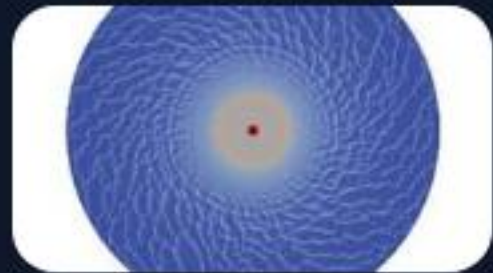
Standardize your codebase. Refactor Top-N. Build for the future.

[oracle.com/sql-standards](https://oracle.com/sql-standards)



# Image Sources

---



[https://static.cambridge.org/content/id/urn%3Acambridge.org%3Aid%3Aarticle%3AS002211202400274X/resource/name/S002211202400274X\\_figAb.png?pub-status=live](https://static.cambridge.org/content/id/urn%3Acambridge.org%3Aid%3Aarticle%3AS002211202400274X/resource/name/S002211202400274X_figAb.png?pub-status=live)

Source: [www.cambridge.org](https://www.cambridge.org)